

# The I2S-pro family: four reasons input-to-state copying gets through

**Backdrop.** *I2S-pro* collects the roadblocks where a fuzzer that *watches the program's comparisons and pastes the observed value back into the input* (cmplog's input-to-state, I2S) gets through, while a byte-mutation fuzzer stays stuck. Picture a lock whose key is lying right next to it: I2S notices the key and uses it; plain fuzzing has to guess. All four categories below share that technique — they differ in *why* it is the decisive one. Counts are validated roadblocks ( $n$ ).

## 1. Exact string / literal gate i2s\_string\_literal\_substitution ( $n = 185$ )

The program demands an exact run of bytes: a magic header, a format tag (IDAT, sRGB), a protocol token (HTTP/, ://). I2S sees that literal in the comparison and drops it straight into the input; random mutation would essentially never spell it out. *The key was lying beside the lock.* This is the dominant case.

```
// lcms  cmspcs.c : _cmsLCMScolorSpace()
// dispatch on the 4-byte ICC color-space signature 'sig'
switch (sig) {
case cmsSigGrayData: return PT_GRAY;    // sig == 'GRAY'
case cmsSigCmykData: return PT_CMYK;    // <-- blocker (lcms/2246): sig ==
    'CMYK'
}
```

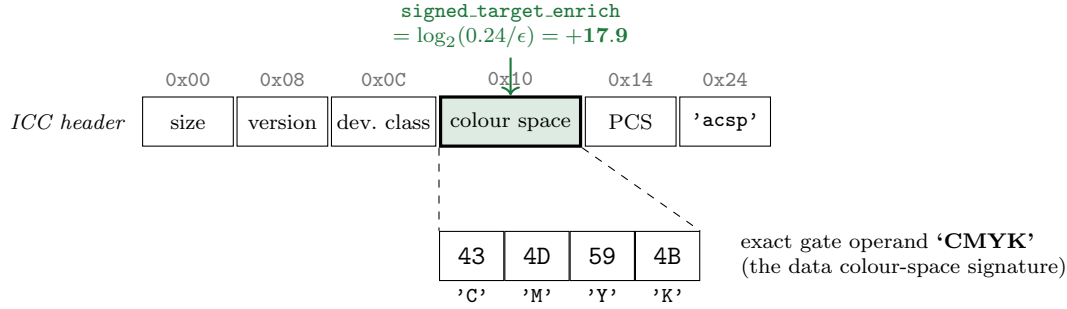
**How we know it is really I2S.** If I2S is planting the literal, the cmplog corpus should be *enriched* in those exact bytes at the gate offset, relative to a naive corpus. We measure that enrichment as a  $\log_2$  frequency ratio; the rule asks for at least a  $2\times$  enrichment.

**Tool:** operand\_enrichment

**Metric:**  $\text{signed\_target\_enrich} = \log_2 \frac{\text{cmplog freq} + \epsilon}{\text{naive freq} + \epsilon}$ ,  $\epsilon=10^{-6}$  ( $> 0$  = enriched,  $< 0$  = depleted)

**Verify rule (the arbiter fires this, no LLM):**  $\text{signed\_target\_enrich} \geq 1.0$

**Real example:** lcms/2246 (\_cmsLCMScolorSpace) — at the operand offset the 4-byte signature is *absent* from the naive corpus (0%) but reaches  $\approx 24\%$  in the cmplog corpus, so  $\text{signed\_target\_enrich} = \log_2(0.24/\epsilon) = +17.9$ ; the decoy bytes are *depleted* ( $\text{decoy\_enrich} = -1.1$ ).



**naive**: 0% of seeds carry 43 4D 59 4B here (a different colour-space)  
**cmplog**:  $\approx 24\%$  carry it — I2SRandReplace splices the logged 'CMYK'

The gate operand of `1cms/2246` is the ICC *data colour-space signature* at offset `0x10`; `operand_enrichment` measures how often each fuzzer's corpus carries the exact bytes *there*. Header field offsets are the ICC format; the measured signal is `signed_target_enrich = +17.9` (naive places the literal in 0% of its seeds), of which  $\approx 24\%$  is the implied cmplog operand-offset rate.

**Sub-case: the literal *floats* (no fixed offset).** Three of these branches place the literal at a *content-dependent* position rather than a fixed file offset — an XML entity name, a comment marker — so the enrichment test above cannot be anchored. The mechanism is identical (paste the exact literal); only the operand's position differs, so we confirm them by the Layer-1 cmplog  $\succ$  naive advantage plus the *absence* of any fixed-offset signature (`gate_signature == false`). Real example (`libxml2, xmlGetPredefinedEntity, entities.c:307`):

```
switch (name[0]) {                                     // name = a parsed entity
    reference
case 'a':                                             // <-- blocker: needs the "amp"
    "/"apos" name
    if (xmlStrEqual(name, "amp")) return &xmlEntityAmp;
    if (xmlStrEqual(name, "apos")) return &xmlEntityApos;
    break;
default: return NULL;                               // naive lands here on all 479
    of its calls
}
```

cmplog takes `case 'a'` 63 times by splicing the logged `"amp"/"apos"` operands into the input; naive never spells them. The literal sits wherever the `&...`; reference occurs, so there is no fixed offset to measure — hence the no-signature confirmation.

## 2. Exact number / enum gate i2s\_numeric\_tag\_substitution ( $n = 39$ )

Same move, but the thing matched is a small *number* — a version code, an enum, a `switch`-case selector — not a string. I2S substitutes the precise value. We split it from category 1 only because “it’s a number” vs “it’s a string” matters when you later try to *recognize* the gate; the mechanism (paste the exact value) is identical.

```
// version/enum dispatch -- gate needs one exact constant
switch (header.version) {
    case 2: parse_v2(...); break; // <-- blocker: only the value 2
    reaches it
}
```

```
default: reject();
}
```

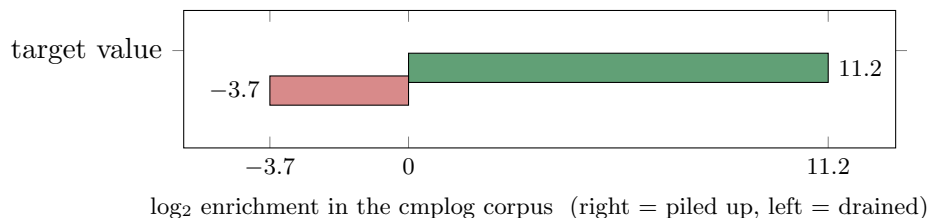
**How we know.** The cmplog corpus over-represents the *target* constant and *under*-represents the competing “decoy” values, exactly as deterministic substitution of one logged operand would predict.

**Tool:** operand\_enrichment

**Metric:** signed\_target\_enrich (target value, log<sub>2</sub> ratio) and decoy\_enrich (the competing value)

**Verify rule (the arbiter fires this, no LLM):** signed\_target\_enrich >= 1.0 AND decoy\_enrich <= 0.0

**Real example:** bloaty/34, switch-case constant — signed\_target\_enrich= 11.2 (target piled up) while decoy\_enrich= -3.7 (wrong values drained).



I2S concentrates the corpus on the one right number and steers it away from the wrong ones.

### 3. Reassembling a structure to reach deeper i2s\_structural\_assembly\_reach\_depth ( $n = 8$ )

Here it is not one literal but several *interlocking* fields (an ELF section table, a PNG iTXt chunk grammar, an XML scheme+attribute). I2S repairs enough of those linked fields that the parser can finally march *deeper* into the input and arrive at the gate. The tell is not “the right bytes are present” — it is “I2S’s seeds drive execution much further in.”

```
// each field must validate before the parser advances to the next
if (read_len(&p) != OK) return;           // field 1
if (read_type(&p) != OK) return;         // field 2 -- I2S repairs these...
if (read_kw(&p) != OK) return;           // field 3
process_text(&p);                          // <-- blocker: only deep, well-
    formed seeds reach it
```

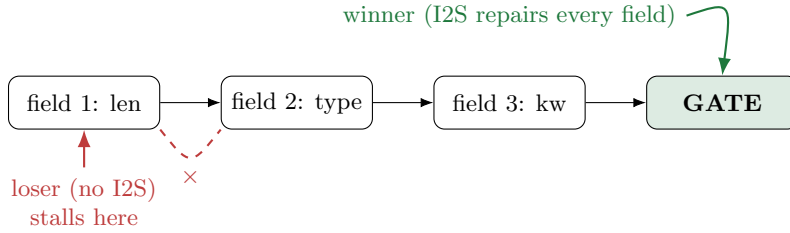
**How we know.** We run each side’s corpus and count, per seed, how many times the blocker line executes (its parse *depth*) and whether the seed takes the decisive side. The winner should reach the line far more.

**Tool:** depth\_reach

**Metric:** depth\_lift = winner / loser per-seed executions of the blocker line; side\_lift = how much more the winner takes the decisive branch; winner\_deeper (bool)

**Verify rule (the arbiter fires this, no LLM):** depth\_lift >= 2.0 AND side\_lift >= 3.0 AND winner\_deeper == true

**Real example:** bloaty/90 (elf.cc:1279, the ELF section-table loop) — depth\_lift= 2375×, side\_lift= 2375: carrying a valid ELF header, the I2S corpus drives the section loop three orders of magnitude deeper than naive.



I2S fixes the interlocking fields so the parser walks the whole chain to the gate; naive dies at the first one.

## 4. Last-mile value precision

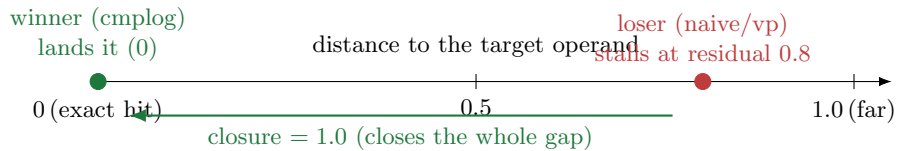
i2s\_operand\_value\_precision ( $n = 6$ )

The sneaky one. *Both* sides *reach* the gate line — even the losing fuzzers get there, sometimes more often. But only the I2S side lands the *exact* required value at the final step; the others get tantalisingly close and never quite hit it. The edge is not *getting* to the door, it is *turning the key precisely* at a door everyone is already standing at.

```
// everyone reaches this line; only the exact operand passes
if (strncmp(line, "Host:", 5) == 0) // loser seeds hit "Host?", "Host
    : ", ...
    route_host(line + 5);           // <-- blocker: needs the precise
    bytes
```

**How we know.** We measure how *close* each side's nearest seed gets to the target operand (a normalised distance, 0 = exact hit). The loser stalls at a positive residual; the winner closes that gap.

**Tool:** value\_distance\_reached  
**Metric:** cmp\_min\_distance = loser's residual to the target (> 0 = never lands it); distance\_closure\_ratio = fraction of the gap the winner closes; distance\_gap, winner\_closer  
**Verify rule (the arbiter fires this, no LLM):** winner\_closer == true AND distance\_gap > 0 AND distance\_closure\_ratio >= 0.5  
**Real example:** curl/129, operand "Host:" — distance\_gap= 0.8, distance\_closure\_ratio= 1.0: the loser stalls 0.8 away, the winner closes the whole gap.



At a glance

| Category                  | <i>n</i>   | Tool / metric                                          | One-line reason                                         |
|---------------------------|------------|--------------------------------------------------------|---------------------------------------------------------|
| string / literal gate     | 185        | operand_enrichment / signed_target_enrich              | paste the exact (incl. 3 floating-gate_signature=false) |
| numeric / enum gate       | 39         | operand_enrichment / signed_target_enrich+decoy_enrich | paste the exact number                                  |
| structural assembly depth | 8          | depth_reach / depth_lift, side_lift                    | rebuild the structure to get                            |
| operand value precision   | 6          | value_distance_reached / distance_closure_ratio        | get the value exactly right finish                      |
| <b>I2S-pro total</b>      | <b>238</b> |                                                        |                                                         |

Counts and example metrics regenerated from `bench/dataset.jsonl`; verify rules are the deterministic arbiter rules (no LLM in the decision).